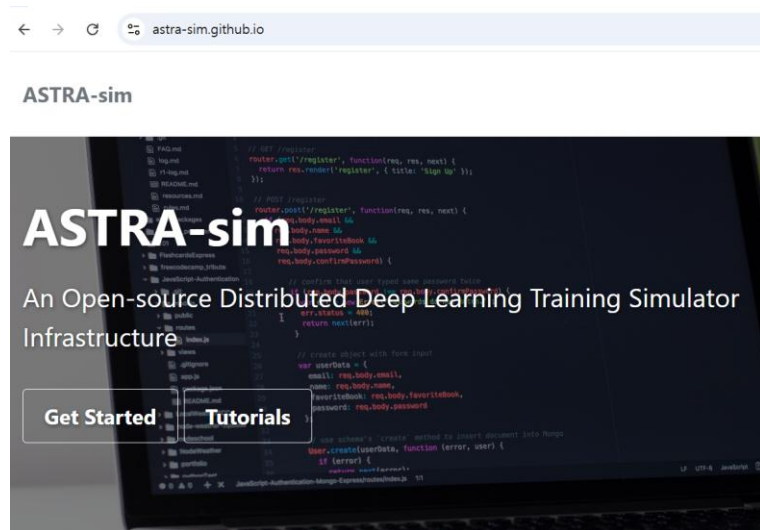

GPU 클러스터 구조와 모델 병렬화 전략에 따른 LLM 분산 추론 성능 비교

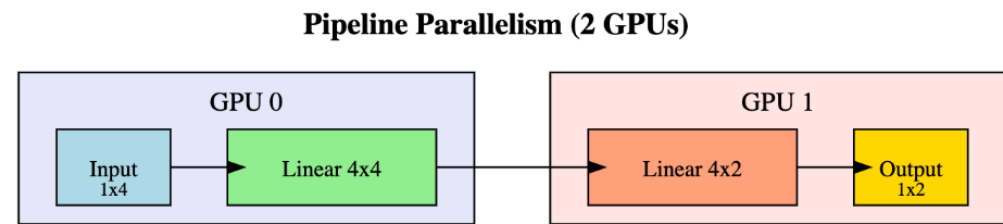
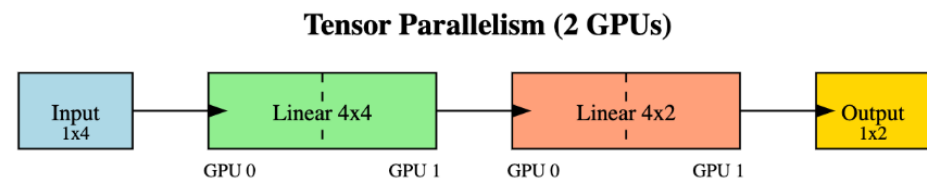
소프트웨어학과 202220735 정서영

프로젝트 소개

- ASTRA-sim을 이용해 다중 GPU 환경을 가정
 - 모델 병렬화 방식(Tensor Parallelism/Pipeline Parallelism)과 GPU 클러스터 구조에 따른 분산 추론 batch_completion_time(ms) 및 throughput(microbatches per sec)비교
- 시뮬레이션 기반 성능 비교 및 분석



ASTRA-sim



Tensor/Pipeline Parallelism

생성형 AI 사용 부분

- ASTRA-sim을 실행하기 위한 시뮬레이션 설정 파일을 작성을 위해 사용
- 각 병렬화 전략에서 발생해야 하는 연산과 통신, 실험에서 가정한 네트워크 bandwidth, 모델 layer 수와 activation 크기 등 실험 configs를 생성형 AI의 입력으로 제공하고 ASTRA-sim의 입력 파일을 작성하는 데 도움을 받음

```
$ run_first_config.sh u X
astra-sim > experiments > first_inode_gpu > $ run_first_config.sh
1 #!/bin/bash
2 set -euo pipefail
3
4 SCRIPT_DIR=$(dirname "$(realpath "$0")")
5 PROJECT_DIR=$(SCRIPT_DIR)/../../
6 ASTRA_SIM=$(PROJECT_DIR)/build/astra_analytical/build/bin/AstraSim_Analytical_Congestion_Aware
7 if [ ! -x "$ASTRA_SIM" ]; then
8   ASTRA_SIM=$(PROJECT_DIR)/build/astra-sim/network_frontend/bin/AstraSim_Analytical_Congestion_Aware
9 fi
10 REMOTE_MEMORY=$(PROJECT_DIR)/examples/remote_memory/analytical/no_memory_expansion.json
11 SYSTEM=$(SCRIPT_DIR)/configs/system_replay_bgpu.json
12 WORKLOAD_DIR=$(SCRIPT_DIR)/workloads
13 RESULT_DIR=$(SCRIPT_DIR)/results
14 RAW_DIR=$(RESULT_DIR)/raw
15 SUMMARY=$(RESULT_DIR)/summary.csv
16
17 mkdir -p "$(WORKLOAD_DIR)" "$(RAW_DIR)"
18
19 cd "$(PROJECT_DIR)"
20
21 echo "[first_inode_gpu] Building analytical backend"
22 "$(PROJECT_DIR)/build/astra_analytical/build.sh"
23
24 echo "[first_inode_gpu] Generating synthetic LLM Chakra traces"
25 python3 "$(SCRIPT_DIR)/scripts/generate_llm_inode_gpu.py" \
26   --output-dir "$(WORKLOAD_DIR)"
27
28 printf "case,parallelism,interconnect,microbatches,wall_time_ns_max,latency_ms,gpu_time_ns_max,gpu_time_ms,comm_time_ns_max,comm_time_ms,logical_comm_bytes,effective_bw_Gbps,throughput_reqs_per_sec,raw_log\n" > "$(SUMMARY)"
29
30 run_case() {
31   local case_name="$1"
32   local parallelism="$2"
33   local interconnect="$3"
34   local microbatches="$4"
35   local workload_prefix="$5"
36   local network_cfg="$6"
37   local raw_log=$(RAW_DIR)/$(case_name).log
38
39   echo "[first_inode_gpu] Running $(case_name)"
40   "$(ASTRA_SIM)" \
41     --workload-configuration=$(WORKLOAD_DIR)/$(workload_prefix) \
42     --system-configuration=$(SYSTEM) \
43     --remote-memory-configuration=$(REMOTE_MEMORY) \
44     --network-configuration=$(network_cfg) \
45     > "$(raw_log)" 2>&1
46
47   local wall
48   local gpu
49   local comm
50   wall=$(awk -F'Wall time:' '{ if ($2+0 > max) max=$2+0 } END { print max+0 }' "$(raw_log)")
51   gpu=$(awk -F'GPU time:' '{ if ($2+0 > max) max=$2+0 } END { print max+0 }' "$(raw_log)")
52   comm=$(awk -F'Comm time:' '{ if ($2+0 > max) max=$2+0 } END { print max+0 }' "$(raw_log)")
53
54   local logical_comm_bytes
55   if [ "$parallelism" = "TPG" ]; then
56     logical_comm_bytes=$(microbatches * 160 * 33554432)
57   else
58     logical_comm_bytes=$(microbatches * 7 * 33554432)
59   fi
60
61   local latency_ms
62   local gpu_ms
63   local comm_ms
64   local effective_bw
65   local throughput
66   latency_ms=$(awk -v ns=$(wall)" "BEGIN { printf "%.2f", ns / 1000000 })
67   gpu_ms=$(awk -v ns=$(gpu)" "BEGIN { printf "%.2f", ns / 1000000 })
68   comm_ms=$(awk -v ns=$(comm)" "BEGIN { printf "%.2f", ns / 1000000 })
69   effective_bw=$(awk -v bytes=$(logical_comm_bytes)" "BEGIN { if (ns > 0) printf "%.2f", bytes / ns; else printf "0.00" })
70   throughput=$(awk -v mb=$(microbatches)" "BEGIN { if (ns > 0) printf "%.2f", mb * 1000000000 / ns; else printf "0.00" })
71
72   printf "%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s\n" \
73     "$(case_name)" "$(parallelism)" "$(interconnect)" "$(microbatches)" \
74     "$(wall)" "$(latency_ms)" "$(gpu)" "$(gpu_ms)" "$(comm)" "$(comm_ms)" \
75     "$(logical_comm_bytes)" "$(effective_bw)" "$(throughput)" "$(raw_log)" \
76     >> "$(SUMMARY)"
77 }
78
```

실험 환경

- 1 node x 8 GPUs

- Node 0: [0,1,2,3,4,5,6,7]

- ① TP8: [0,1,2,3,4,5,6,7]

- ② PP8: [0] -> [1] -> [2] -> [3] -> [4] -> [5] -> [6] -> [7]

- 2 nodes x 4 GPUs

- Node 0: [0,1,2,3], Node 1: [4,5,6,7]

- ① TP8: [0,1,2,3,4,5,6,7]

- ② PP8: [0] -> [1] -> [2] -> [3] -> [7] -> [6] -> [5] -> [4]

- ③ TP4PP2: [0,1,2,3] -> [4,5,6,7]

- ④ TP2PP4: [0,1] -> [2,3] -> [6,7] -> [4,5]

실험 환경

- 4 nodes x 2 GPUs

→ Node 0: [0,1], Node 1: [2,3], Node 2: [4,5], Node 3: [6,7]

① TP8: [0,1,2,3,4,5,6,7]

② PP8: [0] -> [1] -> [3] -> [2] -> [4] -> [5] -> [7] -> [6]

③ TP2PP4: [0,1] -> [2,3] -> [4,5] -> [6,7]

④ TP4PP2: [0,1,2,3] -> [4,5,6,7]

- 8 nodes x 1 GPU

→ Node 0: [0], Node 1: [1], Node 2: [2], Node 3: [3], Node 4: [4], Node 5: [5], Node 6: [6], Node 7: [7]

① TP8: [0,1,2,3,4,5,6,7]

② PP8: [0] -> [1] -> [2] -> [3] -> [4] -> [5] -> [6] -> [7]

실험 환경

- Intra-node NVLink: 400 GB/s, 0.936 us

Intra-node PCIe: 32 GB/s, 1.500 us

Inter-node: 25 GB/s, 5.000 us

- Microbatch size: 1 / 2 / 4 / 8 (1 microbatch = batch size 1인 입력 시퀀스 1개, 2048 tokens)

실험 결과 – NVLink

- 1 node x 8 GPUs

→ 1 node 환경에서는 모든 GPU가 같은 노드 안에 있으므로 통신 비용이 상대적으로 작음

→ 그 결과 **TP8**이 PP8보다 낮은 batch time을 보이며, throughput도 전반적으로 더 높게 나타남

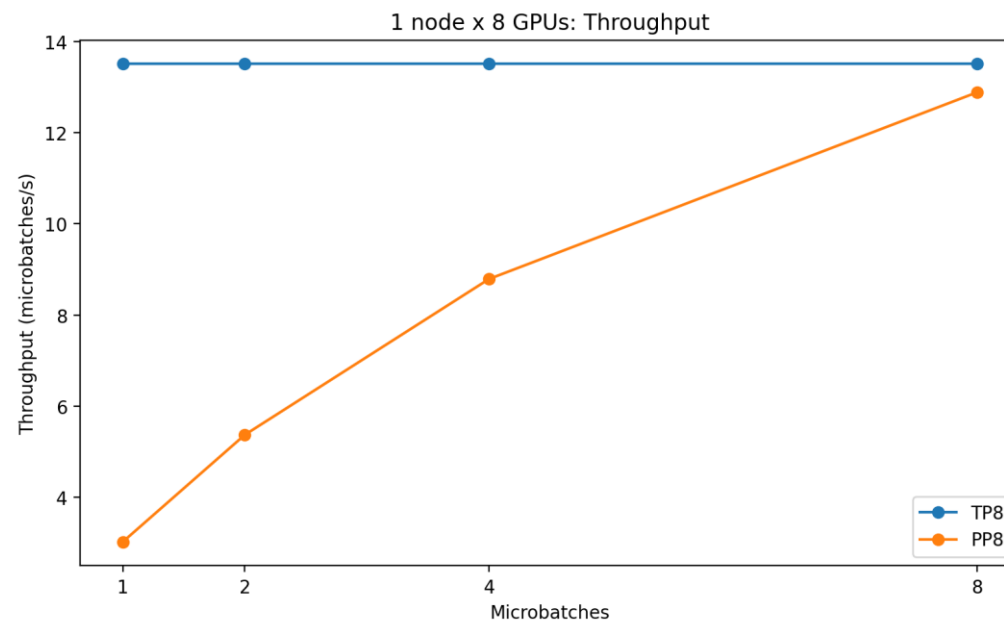
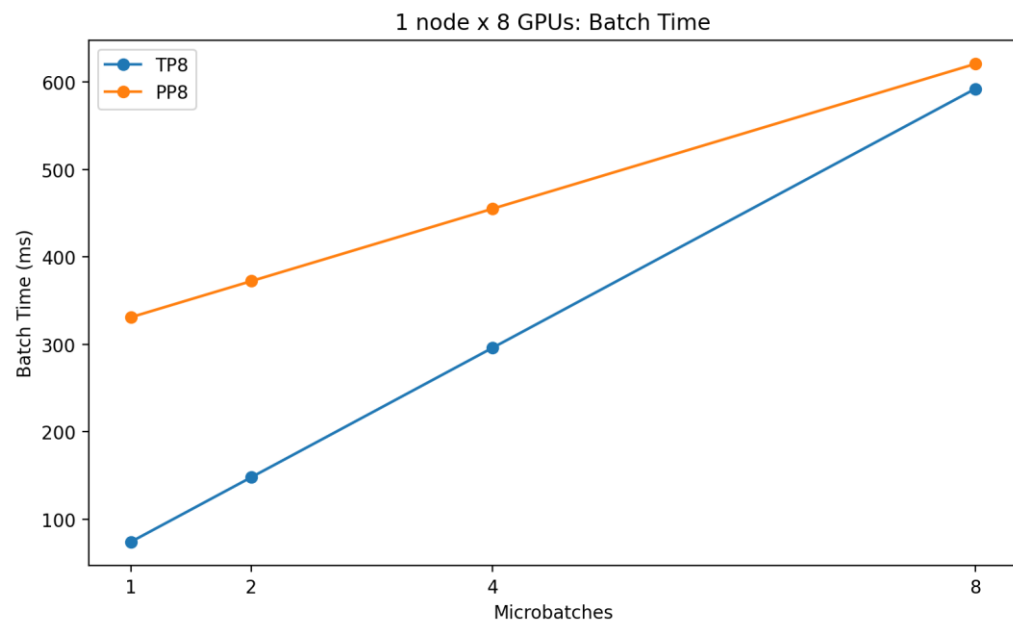


그림 1. NVLink 환경에서 모델 병렬화 전략에 따른 1 node x 8 GPUs 조합의 Batch Time(좌)과 Throughput (우)

실험 결과 – NVLink

- 2 nodes x 4 GPUs

→ 2 nodes 환경에서는 **TP8**과 **TP4PP2**가 비교적 좋은 성능을 보임

→ TP8의 batch time이 가장 낮지만, microbatch 수가 증가할수록 TP4PP2의 throughput도 높아져 hybrid 전략의 장점이 나타남

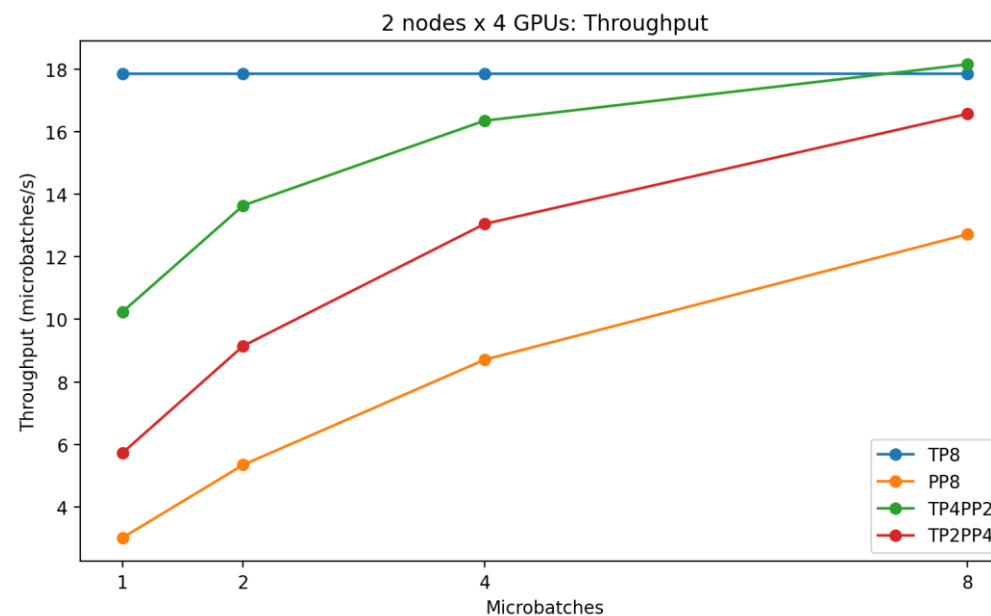
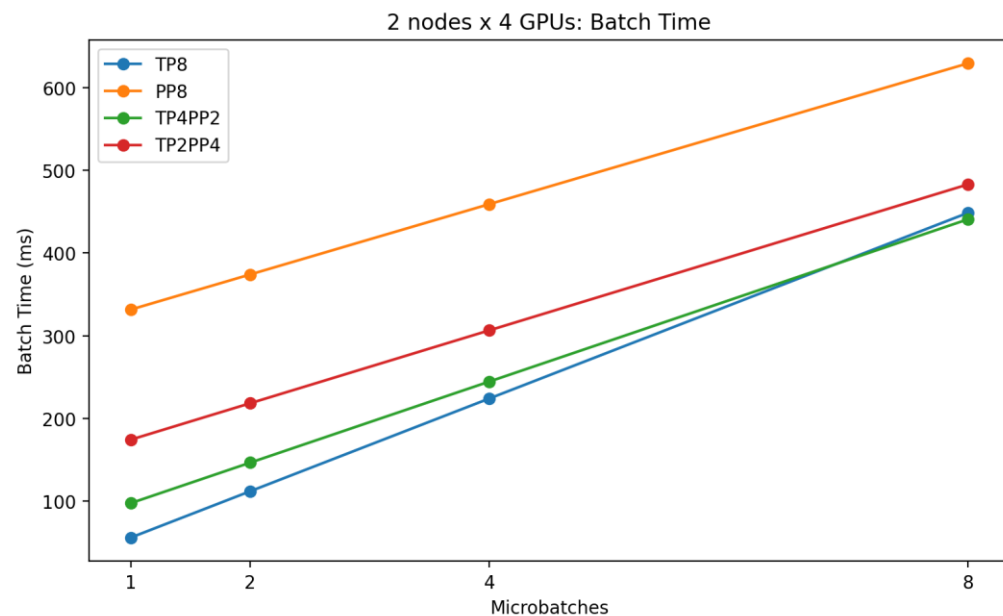


그림 2. NVLink 환경에서 모델 병렬화 전략에 따른 2 nodes x 4 GPUs 조합 Batch Time(좌)과 Throughput (우)

실험 결과 – NVLink

- 4 nodes x 2 GPUs

→ 4 nodes 환경에서도 **TP8**이 가장 낮은 batch time과 높은 throughput을 보임

→ 다만 **TP4PP2** 역시 안정적인 성능을 보이며, TP2PP4보다 더 유리하게 나타남

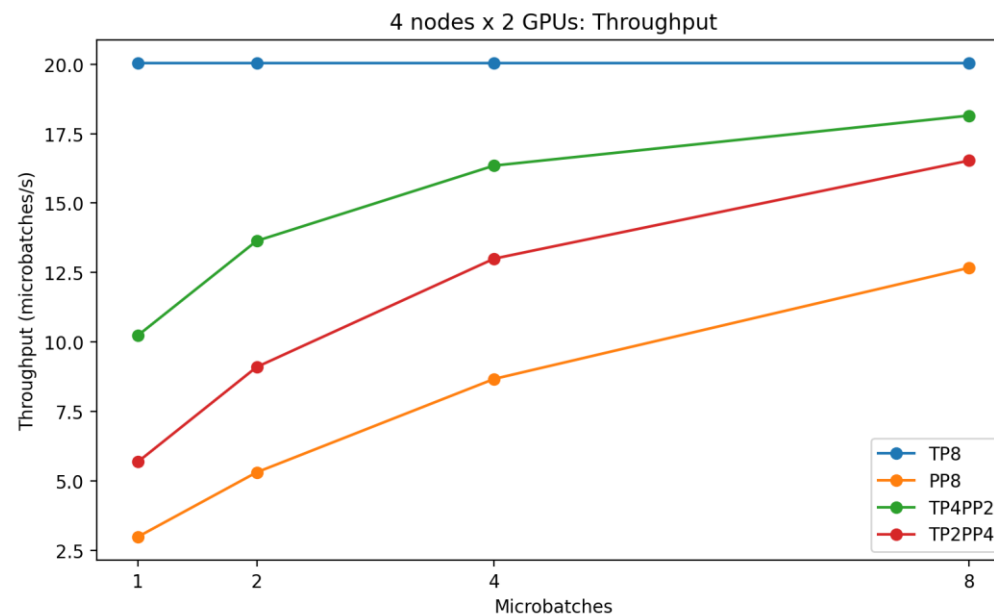
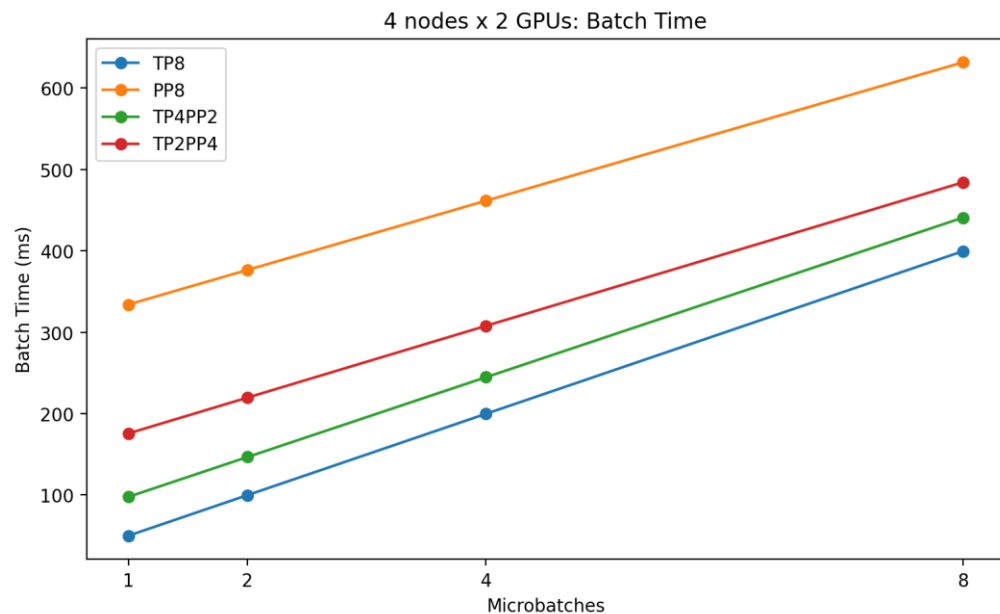


그림 3. NVLink 환경에서 모델 병렬화 전략에 따른 4 nodes x 2 GPUs 조합 Batch Time(좌)과 Throughput (우)

실험 결과 – NVLink

• 8 nodes x 1 GPU

→ TP8에서 발생하는 AllReduce가 모두 노드 간 통신으로 수행되기 때문에 8 node 환경에서는 TP8의 성능이 크게 저하됨

→ 이 경우 **PP8**이 batch time과 throughput 측면에서 더 유리하게 나타난다.

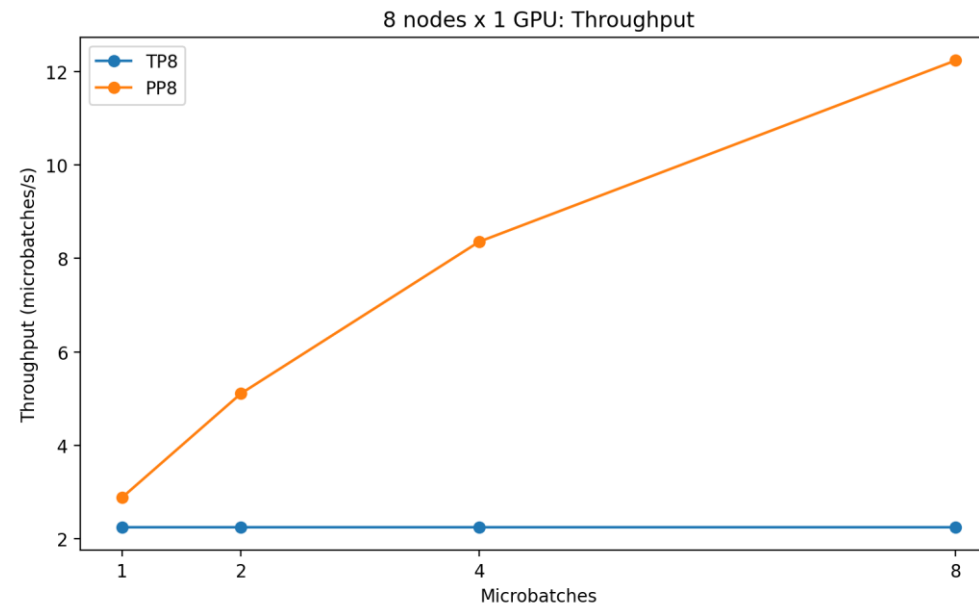
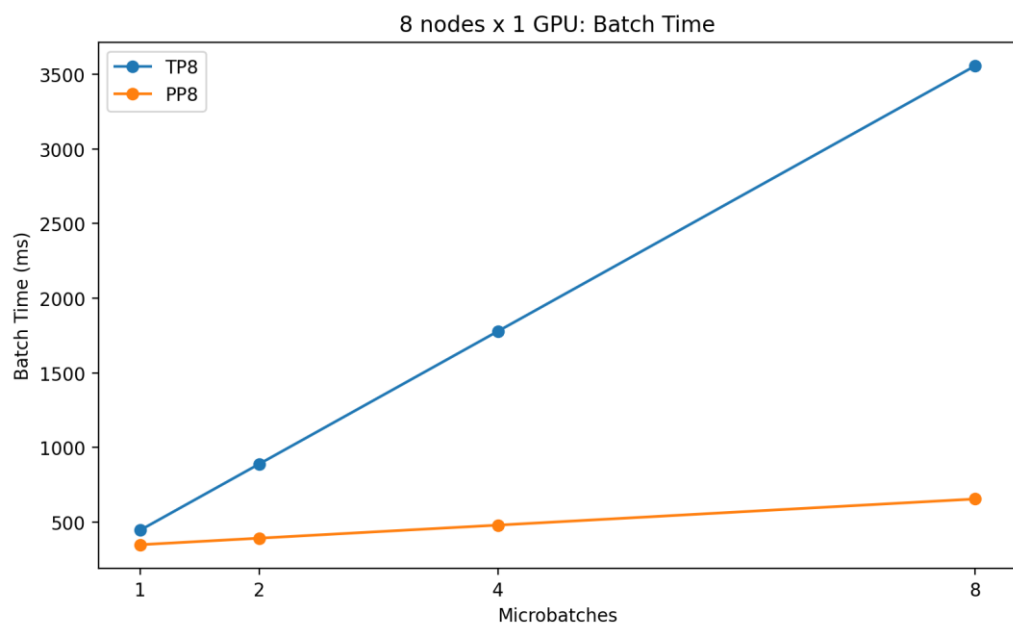


그림 4. NVLink 환경에서 모델 병렬화 전략에 따른 8 nodes x 1 GPU 조합 Batch Time(좌)과 Throughput (우)

결론

- 본 프로젝트에서는 모델 병렬화 방식(Tensor Parallelism/Pipeline Parallelism)과 GPU 클러스터 구조에 따른 분산 추론 시간과 처리량을 시뮬레이션을 이용하여 비교
- LLM 분산 추론 성능은 GPU 수뿐만 아니라 클러스터 구조, 병렬화 전략, GPU 간 통신 비용에 크게 영향을 받음
- 노드 내부 통신이 빠른 환경에서는 TP 기반 전략이 유리하지만,
노드 간 통신 비중이 커질수록 PP 또는 hybrid 전략의 장점이 나타날 수 있음

참고 자료

[1] NVIDIA Developer Blog, "Train Generative AI Models More Efficiently with New NVIDIA Megatron Core Functionalities," NVIDIA, 2024.

Available: <https://developer.nvidia.com/ko-kr/blog/train-generative-ai-models-more-efficiently-with-new-nvidia-megatron-core-functionalities/>

[2] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," arXiv:1909.08053, 2019.

Available: <https://arxiv.org/abs/1909.08053>

[3] Y. Huang et al., "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," arXiv:1811.06965, 2018.

Available: <https://arxiv.org/abs/1811.06965>

[4] ASTRA-sim, "ASTRA-sim: A Distributed Machine Learning System Simulator."

Available: <https://astra-sim.github.io/>

[5] Hugging Face, "Model Parallelism," Transformers Documentation v4.13.0.

Available: <https://huggingface.co/docs/transformers/v4.13.0/parallelism>

감사합니다